

LambLisp Announcement

Contact: Bill White, Frobenius Norm LLC, +1 978 902 0188, white3@frobeniusnorm.com

Frobenius Norm LLC Announces LambLisp™

Real-Time Lisp for Embedded Systems, From AI to Actuator™

LambLisp brings interactive, standards-conformant Lisp to real-time microcontroller applications, and direct integration with local and remote AI accelerators.

Frobenius Norm LLC today announced **LambLisp**, a real-time Lisp implementation for embedded control applications and integration with remote AI accelerators via NVIDIA CUDA and AMD HIP. LambLisp conforms to the Scheme R7RS standard. It has been tested on the Espressif ESP32 family, Linux on x86-64 and aarch64, and with CUDA on the NVIDIA Jetson Orin.

LambLisp Provides Fast and Predictable Real-time Control

Embedded systems typically operate in a control loop: read inputs, calculate, write outputs. To ensure proper operation, an application must exhibit predictable loop times. Traditional dynamic-allocation languages such as Python, MicroPython, and uLisp introduce unpredictable housekeeping pauses (called *garbage collection*, or just *GC*), which are unsuitable for real-time control.

LambLisp divides GC into small increments, eliminating the long pause. Other optimizations have been added, such as using idle time after short loops to do GC in advance; this allows many longer loops to skip GC altogether, reduces average loop time and jitter, and sometimes allows GC to be performed exclusively within otherwise idle periods, when the effective GC overhead becomes zero.

LambLisp also provides an optimizing compiler, reducing the time spent in each loop. Shorter loop times allow the system under control to be operated closer to its physical limits.

LambLisp's native *list* and *dictionary* types provide the same programming convenience as Python's. With its improved GC and easy integration of existing code, LambLisp provides fast and predictable control that Python and its derivatives cannot.

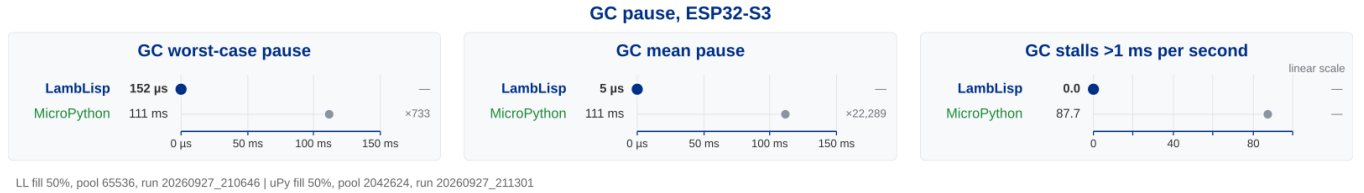


Figure 1 GC Pause: LambLisp Vs. MicroPython



Figure 2 Performance: LambLisp vs. MicroPython

Key Capabilities

Live development on the device. Interact over the serial terminal or network connection. Write and test code immediately, with no recompile-flash-restart cycle. New code takes effect on the next loop iteration. Full firmware updates can be delivered over the air.

A control behavior, written and running on the device with no rebuild:

```
;; typed live at the LL=> prompt on the running robot:
(Sonar.setup)                ; arm the ultrasonic ranger
(define brake 12)             ; a GPIO output pin
(define (watch)               ; called each control-loop pass
  (digitalWrite brake         ; brake if an obstacle is within 20 cm
    (if (< Sonar.latest 0.20) 1 0)))
```

Direct C++ integration. Hardware primitives (GPIO, PWM, I2C, SPI, 1-Wire, WiFi, analog I/O, sonar, camera) are native C++ functions using the same calling convention as LambLisp procedures. LambLisp does not require a separate foreign-function interface or parameter marshaling layer.

One language from sensor to cloud. The same LambLisp code runs on the Espressif ESP32 family, on an x86-64 or aarch64 Linux workstation, and can drive NVIDIA CUDA, cuBLAS, and AMD HIP. Calls to AI coprocessors are made directly when available, or remotely via LLIP (LambLisp Interaction Protocol). There is no language boundary between the real-time controller and remote services.

A full numeric tower. Exact integers widen automatically from 32-bit to 64-bit to arbitrary-precision bignums, alongside rationals and complex numbers. Hardware-assisted cryptography is supported on ESP32 where available.

Libraries for real applications. LambLisp ships typed numeric arrays with N-dimensional indexing and GPU-accelerated matrix multiply (cuBLAS on CUDA builds), a cryptography library (SHA-256, modular exponentiation, RSA, HMAC, HKDF, Diffie-Hellman), and industrial data formats (Modbus TCP and RTU, BACnet/IP, CSV, and binary struct packing).

Standards Conformance. LambLisp implements the Scheme R7RS standard (except `call/cc`), with full tail-call elimination and hygienic `syntax-rules` macros.

Optimized at every level. LambLisp offers high-performance hierarchical dictionaries, used for objects including execution environments. LambLisp also features an optimizing compiler, with per-function runtime choice of bytecode (small), native code (fast), or syntax-tree interpretation with full debug info. Together, the dictionary and compiler give LambLisp the productivity of Python's lists and dictionaries, with the speed and real-time predictability that Python and its derivatives cannot deliver on a microcontroller.

Deterministic and real-time. Bounded, incremental garbage collection and predictable dispatch let control loops meet hard deadlines, so LambLisp suits closed-loop applications where an unannounced scripting-language pause would be unacceptable.

Compact footprint. The runtime and your Scheme application fit comfortably within an ESP32-S3's ~2 MB PSRAM and load from the on-device filesystem, so a deployed controller needs no host tether to run.

Reference Robotic Platform

The reference platform is the **Freenove 4WD autonomous vehicle**, an ESP32-based robot controlling I2C servo and motor drivers (PCA9685), PWM, sonar ranging, digital and analog I/O, and WiFi, all from a live Scheme REPL running at 240 MHz under real-time constraints.

The local system is organized by an **NVIDIA Jetson Orin Nano**. The vehicle's ESP32-S3 runs the control loop; the Jetson carries the perception and coordination work that does not belong on a microcontroller, and the two speak over LLIP, LambLisp's own interaction protocol. The same Scheme runs on both, so work moves between them without a language boundary, a foreign-function interface, or a serialization layer to maintain.

A further node drives a **480x480 RGB mini-HUD**, presenting the local system to an operator: an ESP32-S3 with a parallel-RGB panel carrying the vehicle's live camera view, with the telemetry that matters drawn around it as LVGL widgets – sonar range, motor status, frame timing. The HUD is a peer on the LLIP network rather than a terminal: it subscribes to telemetry the Jetson organizes, and the panel is a LambLisp application like any other, written in Scheme and updated over the air without a refresh.

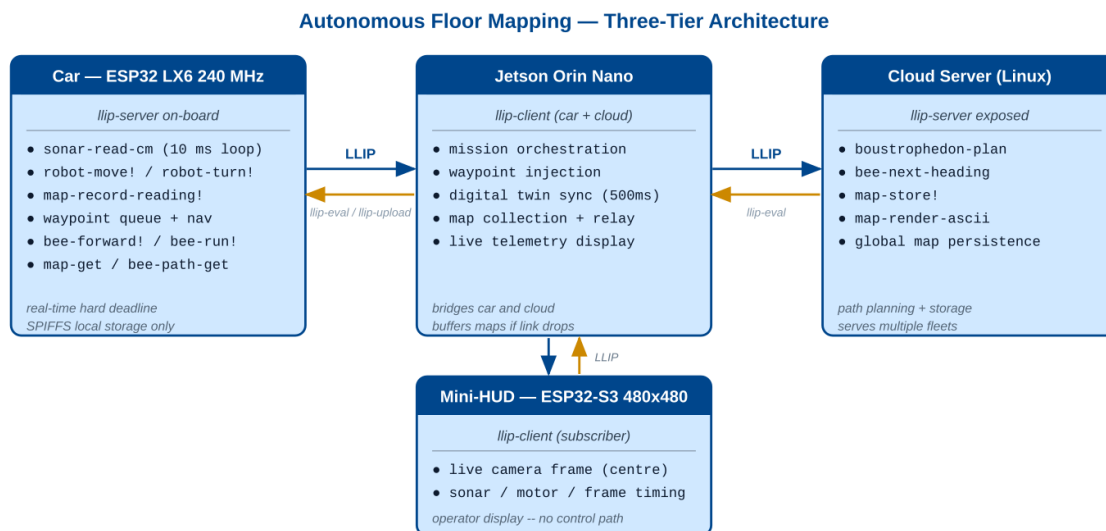


Figure 3 The same LambLisp runtime across three tiers -- the ESP32 vehicle, a local NVIDIA Jetson Orin Nano, and a remote server -- coordinated over LLIP, with the mini-HUD subscribing to the telemetry the Jetson organizes.

Availability and Licensing

LambLisp is in **Alpha**, patent pending, and available now for evaluation and early licensing. Non-commercial use is free. Commercial licensing includes customization, application-development support, and warranty.

- **LambLisp Documentation:** <https://lamblisp.com>
- **Visit the developer:** <https://frobeniusnorm.com>
- **Licensing inquiries:** Bill White, white3@frobeniusnorm.com, +1 978 902 0188

Frobenius Norm LLC develops high-performance control software for physical systems.

Notices

This document is provided for information only. It is not an offer, does not form part of any contract, and creates no warranty of any kind, express or implied. Product descriptions and specifications are subject to change without notice.

Performance and GC-pause figures are measurements of particular builds, boards and workloads, taken at a stated heap occupancy; they characterise those configurations and are not a guarantee of results in any other. Statements about future capability are statements of present intent, not commitments.

LambLisp™ is a trademark of Frobenius Norm LLC. Patent pending. ESP32® and Espressif® are registered trademarks of Espressif Systems (Shanghai) Co., Ltd. NVIDIA®, CUDA® and Jetson® are registered trademarks of NVIDIA Corporation. AMD® and HIP are trademarks of Advanced Micro Devices, Inc. Linux® is a registered trademark of Linus Torvalds. BACnet® is a registered trademark of ASHRAE. Python and MicroPython, uLisp, LVGL and all other marks are the property of their respective owners. Use of a third-party mark here identifies the product only and implies no affiliation with or endorsement by its owner.